

**TRƯỜNG ĐẠI HỌC GIAO THÔNG VẬN TẢI
PHẦN HIỆU TẠI TP.HCM
BỘ MÔN CÔNG NGHỆ THÔNG TIN**



Selenium — Kiểm thử tự động giao diện web (phần 1)

Buổi 10 / 12 — Môn: Kiểm thử Phần mềm

Ngày 1 tháng 8 năm 2026

Nội dung buổi học

- 1 Vị trí trong đề cương — vì sao cần kiểm thử UI tự động
- 2 Selenium là gì: lịch sử, thành phần, kiến trúc WebDriver
- 3 So sánh nhanh Selenium — Cypress — Playwright
- 4 Các bước cài đặt: project test riêng, Maven/Gradle, Selenium Manager
- 5 Chuẩn bị hệ thống thực hành DigiShield (backend + frontend)
- 6 Điều khiển browser: `get`, `getTitle`, `navigate`, `window`
- 7 **Locators**: `By.id`, CSS selector, XPath — chiến lược chọn locator
- 8 Thao tác element: `sendKeys`, `click`, `getText`, `getAttribute`
- 9 Test đăng nhập DigiShield hoàn chỉnh với JUnit 5
- 10 Debug: screenshot, page source, DevTools
- 11 Thực hành và bài nộp

Vị trí trong đề cương môn học

Chương 6 — Công cụ kiểm thử tự động Selenium

Buổi 10 và 11 thuộc **Chương 6** của đề cương: giới thiệu Selenium, khái niệm chung, các bước cài đặt, và xây dựng kịch bản kiểm thử giao diện web tự động.

Chương	Nội dung	Buổi
C4	JUnit, Mockito, integration test	5–7
C5	Kiểm thử hộp đen, API testing (Postman)	8–9
C6	Selenium — kiểm thử UI tự động	10–11
C7	Kiểm thử trong thực tiễn, tổng kết BTL	12

Buổi hôm nay (phần 1)

Cài đặt Selenium WebDriver, tìm element bằng locator, thao tác form và viết test đăng nhập đầu tiên cho DigiShield.

Hành trình kiểm thử DigiShield đến hôm nay (1/2)

Buổi	Tầng	Công cụ	Đối tượng test
5–6	Unit	JUnit 5, Mockito	Interception ServiceImpl, StubAiClient
7	Integration	@SpringBootTest, Testcontainers	RLS đa tenant, sự kiện
8–9	API	Postman, Newman	/api/v1/auth, /interventions
10–11	E2E / UI	Selenium WebDriver	Trang web như người dùng

Hành trình kiểm thử DigiShield đến hôm nay (2/2)

Mảnh ghép cuối cùng

Ta đã test **logic** (unit), **tích hợp** (integration) và **HTTP endpoint** (API).
Tầng còn lại: mô phỏng **người dùng thật** mở browser, chọn vai trò, bấm nút *Đăng nhập*... và kiểm tra hệ thống hoạt động đúng.

Thực trạng DigiShield

Repo đã có module : e2e (Selenium, 1 kịch bản AnalystBlocksAccountE2E) chạy như **cồng chặn** trong CI — nhưng để **đọc hiểu và mở rộng** nó (yêu cầu BTL), hôm nay ta học Selenium từ số 0, tự viết lại bộ test đăng nhập.

Vì sao cần kiểm thử UI tự động?

Chỉ test bằng tay thì sao?

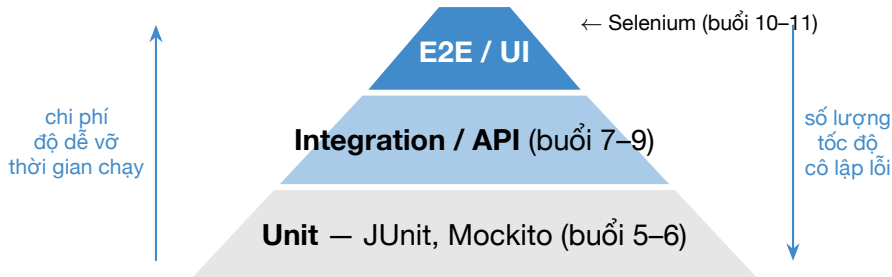
- Mỗi lần release phải bấm lại **toàn bộ** luồng: đăng nhập 4 vai trò, báo cáo phishing, watchlist...
- Lặp đi lặp lại → nhàm chán → **bỏ sót**
- Regression thủ công tốn nhiều giờ mỗi lần sửa code
- Không chạy được trong CI/CD ban đêm

UI test tự động

- Máy “đóng vai” người dùng: mở browser, gõ phím, click
- Chạy lại hàng trăm lần, kết quả **nhất quán**
- Phát hiện lỗi tích hợp frontend-backend mà unit/API test bỏ lọt
- Là lớp **smoke test** trước khi release

Ví dụ lỗi chỉ UI test bắt được: API POST /auth/login trả 200 đúng, nhưng frontend quên điều hướng sang dashboard → API test xanh, người dùng vẫn “kẹt” ở trang login.

Vị trí E2E trên Testing Pyramid



Nguyên tắc

E2E nằm ở **đỉnh** tháp: **ít** test nhất, **đắt** nhất, chậm nhất — nhưng mỗi test bao phủ **cả hệ thống** và mang **giá trị nghiệp vụ cao** nhất.

Đặc điểm của tầng E2E — viết ít nhưng viết đúng chỗ

Vì sao E2E “đắt”?

- Cần **cả hệ thống chạy thật**: backend + frontend + browser
- Chậm: mỗi test vài giây đến vài chục giây
- Giòn: đổi giao diện nhỏ có thể làm vỡ test
- Lỗi khó cô lập: fail ở đâu trong cả chuỗi?

Nên viết E2E cho gì?

- Các luồng **sống còn** (critical path): đăng nhập, báo cáo phishing, chặn giao dịch
- Happy path chính + vài kịch bản tiêu cực quan trọng
- **Không** dùng E2E để kiểm tra mọi biên — việc đó của unit test (EP/BVA buổi 8)

Với DigiShield

Buổi này: luồng **đăng nhập** (cửa ngõ của mọi vai trò). Buổi 11: kịch bản E2E nhiều bước (analyst xử lý báo cáo, watchlist).

Selenium — lịch sử tóm tắt

Năm	Sự kiện
2004	Jason Huggins (ThoughtWorks) tạo Selenium Core — JavaScript tiêm vào trang
2006	Simon Stewart phát triển WebDriver — điều khiển browser “từ bên ngoài”
2009	Hai dự án công bố hợp nhất (Selenium + WebDriver)
2011	Phát hành Selenium 2.0 (Selenium WebDriver)
2018	Giao thức WebDriver trở thành chuẩn W3C chính thức
2021	Selenium 4 : dùng thuần giao thức W3C, bỏ JSON Wire cũ
2022	Selenium 4.6: Selenium Manager — tự động tải driver

Vì sao tên “Selenium”? Đối thủ thời đó là công cụ thương mại Mercury; nguyên tố selen (Selenium) là... thuốc giải độc thủy ngân (Mercury).

Điểm cần nhớ: môn học dùng **Selenium 4.x** + Java — giao thức chuẩn W3C, không cần tải chromedriver thủ công.

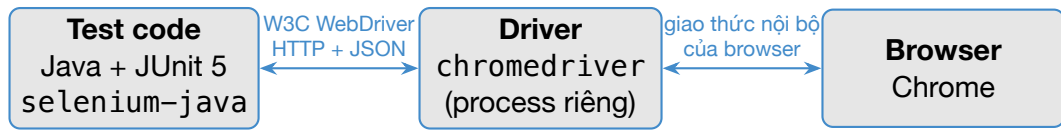
Bộ công cụ Selenium — ba thành phần

Thành phần	Vai trò	Dùng khi
WebDriver	Thư viện lập trình (Java, Python, C#, JS...) điều khiển browser qua giao thức W3C	Viết test tự động trong code — trọng tâm môn học
IDE	Extension Chrome/Firefox: ghi lại thao tác (record & playback), xuất code	Prototype nhanh, người không lập trình
Grid	Phân tán test lên nhiều máy / browser / phiên bản chạy song song	Chạy bộ test lớn trong CI, test đa trình duyệt

Trong môn học

Ta dùng **Selenium WebDriver + Java + JUnit 5** — tận dụng toàn bộ kỹ năng JUnit/AssertJ đã học từ buổi 5. IDE và Grid chỉ cần biết khái niệm.

Kiến trúc Selenium WebDriver (chuẩn W3C)



Selenium Manager tự tải driver đúng phiên bản (từ 4.6)

- Mỗi lệnh (`driver.get(...)`, `click()`...) là một **HTTP request** gửi tới driver; driver dịch sang lệnh riêng của browser.
- Driver do **chính hãng browser** cung cấp: `chromedriver` (Google), `geckodriver` (Mozilla)...
- Nhờ chuẩn W3C, **cùng một test code** chạy được trên Chrome, Firefox, Edge, Safari — chỉ đổi driver.

So sánh nhanh: Selenium — Cypress — Playwright

	Selenium	Cypress	Playwright
Ngôn ngữ	Java, Python, C#, JS, Ruby, Kotlin	JavaScript / TypeScript	JS/TS, Python, Java, .NET
Kiến trúc	Ngoài browser, giao thức chuẩn W3C	Chạy bên trong browser	Ngoài browser, giao thức riêng (CDP)
Browser	Mọi browser chính (driver chính hãng)	Họ Chromium, Firefox, WebKit (hạn chế)	Chromium, Firefox, WebKit
Auto-wait	Không (tự cấu hình wait)	Có sẵn	Có sẵn
Điểm mạnh	Chuẩn hóa, đa ngôn ngữ, cộng đồng lớn nhất, Grid	DX tốt, debug trực quan	Nhanh, API hiện đại

Vì sao môn học chọn Selenium? Theo **đề cương** (Chương 6), là chuẩn W3C, phổ biến nhất trong tuyển dụng tester tại VN; dùng **Java + JUnit 5** — liền mạch buổi 5–7, không học thêm ngôn ngữ mới.

Lời khuyên nghề nghiệp: học vững Selenium → khái niệm locator, wait, Page Object chuyển sang Cypress/Playwright rất nhanh.

Cài đặt — yêu cầu môi trường

Checklist trước khi bắt đầu

- 1 **JDK 17+** (máy đã có — dùng cho DigiShield backend)
- 2 **Google Chrome** bản mới (Selenium Manager sẽ tự khớp driver)
- 3 **IDE:** IntelliJ IDEA / VS Code / Eclipse
- 4 **Maven hoặc Gradle** — buổi này trình bày cả hai
- 5 Mã nguồn DigiShield (backend + frontend) chạy được ở local

Không cần

- **Không** cần tải chromedriver.exe thủ công (Selenium $\geq$ 4.6)
- **Không** cần Docker cho buổi này (backend chạy profile dev với H2)

Bước 1 — Tạo project test riêng

Vì sao project riêng?

E2E test đứng **ngoài** hệ thống, nhìn DigiShield như hộp đen qua browser — giống Postman collection ở buổi 9. Không trộn vào repo backend/frontend.

```
digishield-e2e/  
|-- pom.xml                (hoac build.gradle.kts)  
`-- src  
    |-- test  
    |-- java  
    |-- e2e  
    |   |-- FirstSeleniumTest.java  
    |   |-- LoginTest.java
```

- Toàn bộ code nằm ở `src/test/java` — đây là project “chỉ để test”
- Đặt tên package gợi nghĩa: `e2e`, `e2e.auth`, `e2e.soc...`

Bước 2a — Khai báo dependency với Maven

pom.xml — phần dependencies

```
<dependency>
  <groupId>org.seleniumhq.selenium</groupId>
  <artifactId>selenium-java</artifactId>
  <version>4.27.0</version>
</dependency>
<dependency>
  <groupId>org.junit.jupiter</groupId>
  <artifactId>junit-jupiter</artifactId>
  <version>5.11.4</version>
  <scope>test</scope>
</dependency>
```

- selenium-java: gói tổng, kéo về WebDriver cho mọi browser + Selenium Manager
- junit-jupiter: JUnit 5 quen thuộc từ buổi 5 (assertions, lifecycle)
- Nhớ thêm maven-surefire-plugin bản mới để chạy JUnit 5

Bước 2b — Khai báo dependency với Gradle (Kotlin DSL)

build.gradle.kts — giống cấu trúc DigiShield backend

```
plugins { java }

repositories { mavenCentral() }

dependencies {
    testImplementation(
        "org.seleniumhq.selenium:selenium-java:4.27.0")
    testImplementation(platform(
        "org.junit:junit-bom:5.11.4"))
    testImplementation("org.junit.jupiter:junit-jupiter")
    testRuntimeOnly(
        "org.junit.platform:junit-platform-launcher")
}

tasks.test { useJUnitPlatform() }
```


Bước 3 — Selenium Manager lo phần driver

Trước Selenium 4.6 (cách cũ)

- 1 Xem phiên bản Chrome đang cài
- 2 Tải chromedriver **đúng** phiên bản đó
- 3 Đặt vào PATH hoặc `System.setProperty(...)`
- 4 Chrome tự cập nhật → driver lệch → test vỡ hàng loạt

Từ Selenium 4.6 (cách mới)

- 1 Chỉ cần `new ChromeDriver()`
- 2 **Selenium Manager** (đi kèm trong `selenium-java`) tự phát hiện Chrome, tự tải driver khớp phiên bản, tự cache

Lưu ý khi đọc tài liệu cũ trên mạng

Hướng dẫn bảo tải `chromedriver.exe` hoặc dùng thư viện `WebDriverManager` bên thứ ba là tài liệu **trước 2022** — với Selenium 4.x hiện tại thì không cần nữa.

Bước 4 — Test “khởi động” đầu tiên

FirstSeleniumTest.java

```
import org.junit.jupiter.api.Test;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.chrome.ChromeDriver;

class FirstSeleniumTest {

    @Test
    void moBrowserVaDongLai() {
        // Selenium Manager tu tai driver o lan chay dau
        WebDriver driver = new ChromeDriver();

        driver.get("https://www.selenium.dev");
        System.out.println(driver.getTitle());

        driver.quit(); // lun dong browser khi xong
    }
}
```

Chạy thử — và các lỗi cài đặt thường gặp

Chạy: IDE bấm Run trên method test • Maven `mvn test` • Gradle `./gradlew test`. Kỳ vọng: Chrome bật lên, mở trang, in tiêu đề, tự đóng — test **xanh**.

Triệu chứng	Nguyên nhân / cách xử lý
Lần chạy đầu rất lâu	Selenium Manager đang tải driver — bình thường
<code>NoSuchDriverException</code>	Chưa cài Chrome, hoặc mạng chặn tải driver
Test không được nhận diện	Thiếu JUnit platform / surefire cũ — kiểm tra lại bước 2
Browser mở rồi “đứng” mãi	Quên <code>driver.quit()</code> ở test trước — đóng process thủ công

`quit()` khác `close()`: `close()` chỉ đóng **tab** hiện tại; `quit()` đóng **toàn bộ** browser và kết thúc phiên `WebDriver`. Trong test luôn dùng `quit()`.

Chuẩn bị — chạy backend DigiShield (profile dev)

Terminal 1 — trong thư mục digishield/

```
./gradlew :boot:app:bootRun \  
  --args='--spring.profiles.active=dev'
```

- Profile **dev**: H2 in-memory, **không cần Docker**, dữ liệu demo tự seed
- API sẵn sàng tại `http://localhost:8080/api/v1`
- Dev profile `permitAll` — **không cần tài khoản**; đăng nhập ở frontend chỉ chọn vai trò (slide sau)

Nguyên tắc E2E

Hệ thống đích phải **chạy trước** khi test chạy. Selenium không khởi động ứng dụng hệ ta — nó chỉ điều khiển browser.

Chuẩn bị — chạy frontend DigiShield

Terminal 2 — trong thư mục frontend/

```
npm install      # lan dau
npm run dev      # Vite dev server, port 5173
```

Ghi chú: bản deploy dùng Cognito; `npm run dev` không đặt `VITE_COGNITO_*` nên /login hiện **dev sign-in form** — chính là thứ ta sẽ test.

- Mở thử bằng tay: `http://localhost:5173/login`
- Ô email **tự điền** theo vai trò đang chọn (`learner@dev...`)

Checklist trước khi viết test

- 1 Backend trả dữ liệu? → mở `http://localhost:8080/api/v1`
- 2 Frontend hiển thị trang login? → mở `http://localhost:5173/login`
- 3 Đăng nhập **bằng tay** thành công một lần để hiểu luồng

Đối tượng test hôm nay: trang /login của DigiShield

Các thành phần trên trang (LoginPage.tsx)

- Dropdown **Vai trò** — 6 role RBAC (mặc định learner)
- Ô **Email** — **tự điền theo vai trò** (analyst@dev...); gõ tay thì thôi tự điền
- Nút **Đăng nhập (dev)** (submit) — không kiểm tra mật khẩu; chỉ xuất hiện khi chưa cấu hình Cognito

Điều hướng sau đăng nhập

Vai trò	Trang đích
Learner	/learn
Analyst	/soc/inbox
Super Admin	/super/tenants
3 role còn lại	/dashboard

Assert của test

URL sau đăng nhập **chứa** đường dẫn đích tương ứng.

Nhìn vào DOM của trang login (rút gọn)

```
<form>
  <label for="login-role">Vai trò</label>
  <select id="login-role">
    <option value="super_admin">Super Admin</option>
    ...
    <option value="analyst">Analyst</option>
    <option value="learner" selected>Learner</option>
  </select>
  <label for="login-email">Email</label>
  <input id="login-email" type="email" required
    value="learner@dev.digishield.local" />
  <button id="login-submit" type="submit">
    Đăng nhập (dev)</button>
</form>
```

- Cả 3 element có **id ổn định**: login-role, login-email, login-submit — “hợp đồng” mà module :e2e cũng bám vào
- value của option là **mã role** (analyst) — bền hơn text hiển thị

Điều khiển browser — get, getTitle, getCurrentUrl

```
WebDriver driver = new ChromeDriver();

// Mo trang login cua DigiShield
driver.get("http://localhost:5173/login");

// Tieu de tab hien tai
String title = driver.getTitle();

// URL hien tai - dung de assert dieu huong
String url = driver.getCurrentUrl();
System.out.println(title + " | " + url);

driver.quit();
```

- `get(url)` chờ trang tải xong (sự kiện load) rồi mới trả về
- `getCurrentUrl()` là “mắt” chính để kiểm tra **điều hướng** trong SPA

Điều khiển browser — `manage().window()` và `navigate()`

```
// Kích thước của sổ - UI responsive có thể đổi layout!  
driver.manage().window().maximize();  
driver.manage().window().setSize(  
    new Dimension(1280, 800));  
  
// Điều hướng như người dùng bấm nút trên browser  
driver.navigate().to("http://localhost:5173/login");  
driver.navigate().refresh(); // F5  
driver.navigate().back();    // nút Back  
driver.navigate().forward(); // nút Forward
```

Kinh nghiệm

Luôn cố định kích thước cửa sổ trong `@BeforeEach`: cùng một trang, cửa sổ hẹp có thể ẩn bớt phần tử (menu thu gọn) làm locator tìm không thấy — test “chập chờn”.

WebElement — findElement và findElements

Khái niệm

Locator = “địa chỉ” của một phần tử trong DOM. `findElement(By...)` trả về `WebElement` đầu tiên khớp; không thấy thì ném `NoSuchElementException` (test fail).

```
// Mot element
WebElement email =
    driver.findElement(By.id("login-email"));

// Danh sach element - khong nem exception,
// tra ve list rong neu khong co gi khop
List<WebElement> options =
    driver.findElements(By.tagName("option"));
// Dropdown vai tro co 6 option (6 role RBAC)
```

- `findElements` + kiểm tra `isEmpty()` là cách kiểm tra “element có tồn tại không” mà không làm test nổ

Họ locator By — tổng quan

Locator	Tìm theo	Ví dụ trên DigiShield
By.id	Thuộc tính id	login-role, login-email, login-submit
By.name	Thuộc tính name	(trang login không đặt name)
By.cssSelector	Cú pháp CSS	option[value='analyst']
By.xpath	Biểu thức XPath	nút theo text “Đăng nhập (dev)”
By.linkText / partialLinkText	Text của thẻ <a>	(trang login không có <a> nào!)
By.tagName	Tên thẻ	form, select, option
By.className	Thuộc tính class	(DigiShield style inline, ít class)

Nhận xét từ hệ thống thật: không phải locator nào “trong sách” cũng dùng được — DigiShield không đặt name, ít class, “link” nhiều nơi là span có onClick; tester phải **đọc DOM thật** rồi mới chọn locator.

By.id — locator tốt nhất khi có

```
// Trang login la trang hien hoi cua DigiShield
// co san id on dinh:
WebElement role =
    driver.findElement(By.id("login-role"));
WebElement email =
    driver.findElement(By.id("login-email"));

// Cung tim duoc bang tagName (kem chinh xac hon)
WebElement form =
    driver.findElement(By.tagName("form"));
```

Vì sao id là số một?

- id phải **duy nhất** trong trang (chuẩn HTML)
- Nhanh nhất, dễ đọc nhất, ít vỡ nhất khi giao diện đổi
- Còn gắn với `<label for="...">` → tốt cả cho accessibility

CSS selector — cú pháp cần nhớ

Cú pháp	Ý nghĩa
#login-email	Element có id login-email
.ten-class	Element có class
button	Mọi thẻ button
button[type='submit']	button có thuộc tính type="submit"
input[placeholder*='x']	placeholder chứa chuỗi x (^= bắt đầu, \$= kết thúc)
form > button	button là con trực tiếp của form
form button	button là hậu duệ (mọi cấp) của form
button:nth-of-type(3)	Nút thứ 3 cùng cha (để vỡ — hạn chế dùng)

Mẹo: CSS selector chính là thứ các bạn đã dùng khi viết CSS — và có thể **thử ngay trong DevTools** trước khi đưa vào test.

CSS selector — ví dụ thật trên DigiShield

```
// Nut submit duy nhất trong form login
driver.findElement(
    By.cssSelector("button[type='submit']"));

// Một option cụ thể trong dropdown vai trò
driver.findElement(By.cssSelector(
    "#login-role option[value='analyst']"));

// Trang Watchlist (/soc/watchlist): ô Quick Check
// không có id, nhưng có placeholder đặc trưng
driver.findElement(By.cssSelector(
    "input[placeholder*='domain']"));
```

- Placeholder thật của ô Quick Check (trang Watchlist): “*Nhập số tài khoản, SĐT, hoặc domain...*” — ta khớp đoạn 'domain' trong đó
- *= khớp **một phần** chuỗi — đỡ vỡ khi câu chữ chỉnh nhẹ, nhưng vẫn phụ thuộc ngôn ngữ hiển thị

XPath — cú pháp cần nhớ

Cú pháp	Ý nghĩa
<code>//button</code>	Mọi button ở bất kỳ đâu
<code>//input[@id='login-email']</code>	input có thuộc tính id
<code>//option[text()='Analyst']</code>	Text bằng đúng “Analyst”
<code>//button[contains(text(),'Đăng')]</code>	Text chứa chuỗi con
<code>//form//button</code>	button bên trong form
<code>//label[@for='login-role']/..</code>	Đi ngược lên cha (CSS không làm được)
<code>(//button)[2]</code>	Nút thứ 2 trong kết quả (dễ vỡ!)

Khi nào cần XPath?

Khi phải tìm theo **text hiển thị** hoặc đi **ngược cây DOM** — hai việc CSS selector không hỗ trợ. Còn lại, ưu tiên CSS (ngắn gọn, nhanh hơn).

XPath theo text — và cái bẫy tiếng Việt

```
// Nut submit theo text hiện thi
driver.findElement(By.xpath(
    "//button[contains(text(),'Đăng nhập')]"));

// Option theo text hiện thi (nhân tiếng Anh:
// 'Super Admin', 'Analyst', 'Learner'...)
driver.findElement(By.xpath(
    "//option[text()='Analyst']"));
```

Ba cái bẫy khi tìm theo text trên DigiShield

- ❶ Text có **dấu tiếng Việt** — file test phải lưu UTF-8, gõ đúng dấu
- ❷ Cùng trang /login, **hai chế độ**: bản dev nút ghi “Đăng nhập (dev)”, bản Cognito ghi “Đăng nhập” — text khác, id mới là chỗ bấm bên
- ❸ DigiShield có i18n (t('Đăng nhập (dev) ')) — đổi ngôn ngữ giao diện là test vỡ

Chiến lược chọn locator: id > CSS > XPath

Ưu tiên	Locator	Ưu điểm	Nhược điểm
1	By.id (hoặc data-testid)	Duy nhất, bền, nhanh	Cần dev đặt sẵn
2	By.cssSelector	Ngắn gọn, quen thuộc, nhanh	Không tìm theo text
3	By.xpath	Mạnh nhất (text, đi ngược cây)	Dài, dễ vỡ, chậm hơn
Tránh	Vị trí tuyệt đối, nth-child, class tự sinh	—	Vỡ ngay khi UI đổi

Quy tắc vàng: locator tốt = **bền với thay đổi giao diện** + **đọc là hiểu ngay**.

Đừng bao giờ copy nguyên XPath tuyệt đối

/html/body/div[2]/div/div[3]/... từ DevTools vào test!

Thực trạng DigiShield — khi hệ thống “không chiều” tester

Quan sát từ mã nguồn frontend

- Trang login có id ổn định đủ bộ (login-role/-email/-submit); Alert Center có vài id lẻ (alert-severity, alert-message)
- Đa số trang khác (Dashboard, SOC Inbox, Watchlist...): input dùng **style inline, không id, không name**, ít class
- Nút bấm chỉ phân biệt được bằng text hoặc vị trí

Chiến lược ngắn hạn (tuần này)

Dựa vào thuộc tính **có nghĩa** đang tồn tại: type, placeholder (SOC Inbox), text nút, cấu trúc form.

Chiến lược dài hạn (đúng bài)

Đề xuất dev thêm data-testid vào các element quan trọng — hợp tác dev-tester, chi phí thấp, lợi ích lâu dài.

data-testid — best practice hợp tác dev–tester

Đề xuất pull request cho các trang SOC (chưa có id)

```
<input placeholder="Nhập số tài khoản..."
        data-testid="watchlist-quick-check" />
<button data-testid="watchlist-add">
```

```
// Trong test: chỉ bấm vào "hộp đồng" testid
driver.findElement(By.cssSelector(
    "[data-testid='watchlist-quick-check']"));
```

Vì sao đáng làm? (1) Tách locator khỏi style/text: refactor CSS, đổi câu chữ, i18n **không** làm vỡ test; (2) là “hợp đồng” rõ ràng giữa dev và tester — đổi testid là biết phải sửa test; (3) không ảnh hưởng người dùng cuối (thuộc tính data-* tro với browser).

Nhập liệu — sendKeys và clear

```
WebElement email =  
    driver.findElement(By.id("login-email"));  
  
// CAN THAN: o email da duoc tu dien san  
// (learner@dev.digishield.local). sendKeys  
// chi GO THEM vao noi dung cu  
// -> phai clear() truoc!  
email.clear();  
email.sendKeys("tester@digishield.vn");
```

Lỗi kinh điển với form có prefill

Quên clear() → ô email thành
learner@dev.digishield.localtester@digishield.vn — test
fail khó hiểu.

Kích hoạt — click và Select cho dropdown

```
// Click nút submit (khuyen dung – giống het  
// hành vì người dung)  
driver.findElement(By.id("login-submit")).click();  
  
// Dropdown KHÔNG click như nút – dùng lớp Select:  
Select role = new Select(  
    driver.findElement(By.id("login-role")));  
role.selectByValue("analyst");           // theo mã role  
role.selectByVisibleText("Analyst");     // theo nhãn
```

- `click()` yêu cầu element **nhìn thấy được và không bị che**; bị che thì ném `ElementClickInterceptedException`
- `Select` (gói `support.ui`) là API riêng cho `<select>`; ưu tiên `selectByValue` — mã role không đổi khi giao diện đổi nhãn/ngôn ngữ

Đọc thông tin — getText và getAttribute

```
WebElement submit =  
    driver.findElement(By.id("login-submit"));  
  
String label = submit.getText();  
// -> "Đăng nhập (dev)"  
String type = submit.getAttribute("type");  
// -> "submit"  
  
WebElement email =  
    driver.findElement(By.id("login-email"));  
// Voi <input>, noi dung go vao KHONG nam  
// trong getText() ma trong thuoc tinh value:  
String value = email.getAttribute("value");  
// -> "learner@dev.digishield.local" (tu dien)
```

Ghi nhớ

`getText()` = text **hiển thị** bên trong thẻ (nút submit trả về “Đăng nhập (dev)”; còn giá trị của `<input>` phải đọc qua `getAttribute("value")`).

Kiểm tra trạng thái — `isDisplayed`, `isEnabled`, `isSelected`

Method	Trả về true khi
<code>isDisplayed()</code>	Element đang hiển thị (không <code>display:none</code> /ẩn)
<code>isEnabled()</code>	Element không bị disabled
<code>isSelected()</code>	Checkbox/radio/option đang được chọn

```
// isSelected: dùng chuan cho <option>
new Select(driver.findElement(
    By.id("login-role"))).selectByValue("analyst");

WebElement analystOpt = driver.findElement(
    By.cssSelector("option[value='analyst']"));
analystOpt.isSelected(); // true - vừa chọn xong

WebElement learnerOpt = driver.findElement(
    By.cssSelector("option[value='learner']"));
learnerOpt.isSelected(); // false
```

Ứng dụng: `assert trang thái form` trước khi submit — chọn nhầm role thì fail

Bước 1 — đặc tả test case (khuôn mẫu buổi 9)

Mã	Mục tiêu	Dữ liệu	Kết quả mong đợi
TC-L01	Đăng nhập Admin	Org role org_admin, email tự điền	Sang /dashboard
TC-L02	Analyst thấy báo cáo	hộp role analyst	/soc/inbox + tab THREAT
TC-L03	Email sai định dạng	khong-phai-email	Ở lại /login
TC-L04	F5 sau khi đăng nhập	đăng nhập rồi F5	Giữ phiên? (slide sau!)

Quy trình không đổi

Thiết kế test case (hộp đen — buổi 8–9) **trước**, rồi mới tự động hóa bằng Selenium. Công cụ đổi, tư duy giữ nguyên.

Bước 2 — khung JUnit test class

```
class LoginTest {  
  
    static final String LOGIN_URL =  
        "http://localhost:5173/login";  
  
    WebDriver driver;  
  
    @BeforeEach  
    void setUp() {  
        driver = new ChromeDriver();  
        driver.manage().window().setSize(  
            new Dimension(1280, 800));  
        driver.get(LOGIN_URL);  
    }  
  
    @AfterEach  
    void tearDown() { driver.quit(); }  
}
```

Bước 3 — TC-L01: đăng nhập Org Admin

```
@Test
void dangNhapOrgAdmin_veDashboard() {
    new Select(driver.findElement(
        By.id("login-role")))
        .selectByValue("org_admin");
    // email đã điền theo role - không cần gõ

    driver.findElement(By.id("login-submit")).click();

    assertTrue(driver.getCurrentUrl()
        .contains("/dashboard"));    // XANH ngay!
}
```

Vì sao assert URL ngay sau `click()` vẫn xanh?

Handler submit của React gọi `navigate()` **đồng bộ** ngay trong sự kiện `click` — URL đã đổi trước khi `click()` trả về. Nhưng đừng vội mừng... URL không phải là tất cả (slide sau).

TC-L02: lúc ĐỎ lúc XANH?! — test “chập chờn”

```
@Test
void dangNhapAnalyst_thayHopBaoCao() {
    new Select(driver.findElement(By.id("login-role")))
        .selectByValue("analyst");
    driver.findElement(By.id("login-submit")).click();

    assertTrue(driver.getCurrentUrl()
        .contains("/soc/inbox"));           // luôn XANH
    driver.findElement(By.xpath(
        "//button[@role='tab'][contains(., 'THREAT')]"));
    // lan dau: NoSuchElementException - ĐỎ
    // chay lai: co khi lai XANH ?!
}
```

- URL đổi **đồng bộ**; nội dung trang đích (lazy(() => import...) + gọi API) đến **sau**
- Test **đua** với trang render: thắng xanh, thua đỏ — *flaky test*, nguy hiểm hơn test đỏ hăn (buổi 11)

Giải pháp tạm thời — và vấn đề để lại cho buổi 11 (1/2)

```
@Test
void dangNhapAnalyst_thayHopBaoCao()
    throws InterruptedException { // <-- nho them throws!
    new Select(driver.findElement(By.id("login-role")))
        .selectByValue("analyst");
    driver.findElement(By.id("login-submit")).click();

    // TAM THOI: cho "cung" 1.5 giay
    Thread.sleep(1500);

    driver.findElement(By.xpath(
        "//button[@role='tab'][contains(., 'THREAT')]"));
    // tim thay -> XANH
}
```

Giải pháp tạm thời — và vấn đề để lại cho buổi 11 (2/2)

Thread.sleep là anti-pattern

- Máy nhanh: **lãng phí** 0.9s mỗi test × hàng trăm test
- Máy chậm / CI quá tải: 1.5s vẫn **không đủ** → test chập chờn (flaky)
- Con số 1500 là “bói” — không gắn với điều kiện thật nào

Câu hỏi treo cho Buổi 11

Làm sao chờ **đúng sự kiện** “element đã xuất hiện” thay vì chờ thời gian?
→ WebDriverWait + ExpectedConditions (explicit wait).

Gom điểm chung — helper dangNhap(role)

```
void dangNhap(String role) {  
    new Select(driver.findElement(By.id("login-role")))  
        .selectByValue(role);  
    driver.findElement(By.id("login-submit")).click();  
}  
  
@Test  
void dangNhapLearner_veLearn()  
    throws InterruptedException {  
    dangNhap("learner");  
    Thread.sleep(1500);  
    assertTrue(driver.getCurrentUrl()  
        .contains("/learn"));  
}
```

Mỗi role một dòng `dangNhap("...")` — hết lặp code. Đây chính là bước đệm cho **Page Object Model** (buổi 11): gom locator + thao tác của một trang vào một class.

TC-L03 — kịch bản tiêu cực: email sai định dạng

```
@Test
void emailSaiDinhDang_oLaiTrangLogin()
    throws InterruptedException {
    WebElement email =
        driver.findElement(By.id("login-email"));
    email.clear();
    email.sendKeys("khong-phai-email");
    driver.findElement(By.id("login-submit")).click();
    Thread.sleep(500);

    assertTrue(driver.getCurrentUrl()
        .contains("/login"));    // XANH
}
```

Vì sao vẫn ở /login?

type="email" required → **HTML5 constraint validation** chặn submit; handler React không chạy. Message (đọc qua `getAttribute("validationMessage")`) do browser sinh — đừng assert nội dung!

TC-L04 — F5 sau đăng nhập: test tìm ra “chuyện thú vị”

```
@Test
void f5SauDangNhap_giuPhien()
    throws InterruptedException {
    dangNhap("analyst");
    Thread.sleep(1500);

    driver.navigate().refresh();    // F5
    Thread.sleep(1000);
    // Ky vong: van o /soc/inbox. Thuc te: D0!
    assertTrue(driver.getCurrentUrl()
        .contains("/soc/inbox"));    // dang o /login
}
```

Test fail — nhưng là phát hiện giá trị: phiên dev nằm **trong bộ nhớ JS** — F5 là mất, app về /login. By design: bản Cognito lưu phiên trong `localStorage`, F5 **giữ** phiên.

Luật rút ra: giữa kịch bản **không** `driver.get()` (= reload = văng phiên) — điều hướng **qua UI**. Đây là lý do POM của module : e2e click sidebar (buổi 11).

Debug — chụp screenshot khi test fail

```
import org.openqa.selenium.OutputType;
import org.openqa.selenium.TakesScreenshot;

// Ep kieu driver sang TakesScreenshot
File shot = ((TakesScreenshot) driver)
    .getScreenshotAs(OutputType.FILE);

Files.copy(shot.toPath(),
    Path.of("build/loi-dang-nhap.png"),
    StandardCopyOption.REPLACE_EXISTING);
```

Vì sao cần?

Test E2E fail trong CI lúc 2 giờ sáng — không ai nhìn thấy browser. Screenshot tại thời điểm fail là **bằng chứng** quan trọng nhất để chẩn đoán (trang trắng? lỗi 403? nút bị che?).

Nâng cao (tự tìm hiểu): gói chụp màn hình vào @AfterEach chỉ khi test fail (JUnit TestWatcher) — sẽ gặp lại ở buổi 11–12 khi nói về CI.

Debug — page source và DevTools

```
// Toàn bộ HTML tại thời điểm gọi -  
// xem element có thực sự tồn tại không  
String html = driver.getPageSource();  
System.out.println(  
    html.contains("login-email")); // true?
```

Quy trình tìm selector bằng Chrome DevTools

- 1 Mở trang → F12 (hoặc chuột phải → Inspect)
- 2 Bấm biểu tượng mũi tên, trở vào element cần tìm
- 3 Đọc DOM: có id? có thuộc tính đặc trưng?
- 4 Thử selector ngay trong Console:
document.querySelector("...") hoặc Ctrl+F trong tab Elements (dán được cả XPath)
- 5 Chỉ khi selector “ăn” đúng 1 element mới đưa vào test

Tổng hợp — bộ kỹ năng Selenium sau buổi hôm nay

Nhóm	API đã học
Vòng đời Điều khiển browser	<code>new ChromeDriver()</code> , <code>quit()</code> (so với <code>close()</code>) <code>get</code> , <code>getTitle</code> , <code>getCurrentUrl</code> , <code>navigate()</code> , <code>manage().window()</code>
Tìm element	<code>findElement(s) + By.id / cssSelector /</code> <code>xpath / tagName...</code>
Thao tác	<code>sendKeys</code> , <code>clear</code> , <code>click</code> , <code>Select.selectByValue</code>
Đọc / kiểm tra	<code>getText</code> , <code>getAttribute</code> , <code>isDisplayed</code> , <code>isEnabled</code>
Debug	<code>TakesScreenshot</code> , <code>getPageSource</code> , <code>DevTools</code>

Điểm yếu còn lại của bộ test hôm nay: `Thread.sleep` rải khắp nơi. Buổi 11 sẽ thay bằng **explicit wait**, gom locator vào **Page Object Model**, và chạy **headless** cho CI.

Thực hành trên lớp

Bài 1 — Dựng và chạy bộ test đăng nhập (bắt buộc)

Tạo project `digishield-e2e-btl` (Maven hoặc Gradle), chạy backend dev + frontend, hoàn thiện `LoginTest` với 3 test **xanh**: Org Admin (TC-L01), Analyst + tab THREAT (TC-L02), email sai định dạng (TC-L03) — và TC-L04 kèm **nhận xét** vì sao đỏ.

Bài 2 — Mở rộng vai trò

Dùng helper `dangNhap(role)`, viết thêm 3 test: `learner` → `/learn`, `super_admin` → `/super/tenants`, `content_editor` → `/dashboard`.

Bài 3 — Trình sát locator cho buổi 11

Đăng nhập Analyst, dùng DevTools tìm selector cho: ô tìm kiếm SOC Inbox (theo placeholder), một dòng trong danh sách báo cáo. Chụp screenshot sau đăng nhập bằng `TakesScreenshot` và lưu file.

Bài nộp (về nhà — gắn với Bài tập lớn)

Yêu cầu: bộ kịch bản Selenium cho chức năng đăng nhập

Áp dụng cho **ứng dụng của nhóm** (BTL) — nhóm nào dùng DigiShield thì làm trên DigiShield:

- 1 Bảng **đặc tả test case** đăng nhập theo khuôn mẫu buổi 9: tối thiểu 5 ca (2 tích cực, 3 tiêu cực)
- 2 Project test riêng (Maven/Gradle) cài Selenium 4.x + JUnit 5, tự động hóa đủ 5 ca, có @BeforeEach/@AfterEach
- 3 Nhận xét về **locator** của ứng dụng nhóm: element nào thiếu id ổn định, đề xuất data-testid ở đâu
- 4 README.md: cách chạy hệ thống + cách chạy test

Nộp

Đẩy lên repo nhóm trước **buổi 11**. Đây là thành phần “Selenium E2E” trong báo cáo BTL (A1.2 — 30% điểm học phần).

Tổng kết và buổi tới

Hôm nay đã học:

- Vị trí E2E trên testing pyramid: ít — đắt — giá trị cao
- Selenium: lịch sử, WebDriver/IDE/Grid, kiến trúc W3C
- Cài đặt: selenium-java 4.x + JUnit 5, Selenium Manager
- Locators: ưu tiên id > CSS > XPath; data-testid
- Bộ test đăng nhập DigiShield 4 kịch bản

Buổi 11 — Selenium (phần 2)

- **Explicit wait:** WebDriverWait, ExpectedConditions — xóa sổ Thread.sleep
- **Page Object Model** — tổ chức code test
- Kịch bản E2E nhiều trang (SOC Inbox, Watchlist)
- Chạy **headless** chuẩn bị cho CI

Chuẩn bị

Hoàn thành bài nộp; giữ project digishield-e2e chạy được.

Tài liệu tham khảo

- [1] *Slide bài giảng điện tử môn Kiểm thử Phần mềm.*
- [2] Paul Ammann, Jeff Offutt, *Introduction to Software Testing*, Cambridge University Press, 2008.
- [3] Glenford J. Myers, *The Art of Software Testing*, 2nd ed., Wiley, 2004.
- [4] ISTQB Foundation Level Syllabus, version 4.0, 2023. <https://istqb.org/>
- [5] Selenium Project, *Selenium Documentation*, <https://www.selenium.dev/documentation/>.
- [6] W3C, *WebDriver Specification*, <https://www.w3.org/TR/webdriver/>.
- [7] Mã nguồn thực hành DigiShield: frontend (`LoginPage.tsx`, `router.tsx`) và backend Spring Modulith.